

Autonomous Systems:

Xây dựng Autonomous Agent hoàn thành checklist ba bước

Trong làn sóng phát triển của Trí tuệ Nhân tạo hiện đại, khái niệm **AI Agent** đã có bước chuyển mình ngoạn mục từ những chuỗi câu lệnh tĩnh (Chains/Workflows) sang các hệ thống tự hành (**Autonomous Systems**). Một tác nhân tự hành hướng mục tiêu không chỉ thực thi một chuỗi hướng dẫn cố định được lập trình sẵn mà sở hữu khả năng tự phân tích mục tiêu (Goal-driven Planning), tự chia nhỏ nhiệm vụ thành các bước khả thi, tuân tự thực thi, truyền dữ liệu ngữ cảnh giữa các chặng và tự đánh giá để kích hoạt điều kiện dừng (**Stop Condition**) an toàn.

Bài học thực hành chuyên sâu này sẽ hướng dẫn bạn xây dựng trọn vẹn một **Autonomous Checklist Agent** với giới hạn hữu hạn (**Bounded Loop**) tối đa ba bước. Chúng ta sẽ lấy kịch bản thực tế rất trực quan và gần gũi: *Người dùng nhập yêu cầu viết một bài chia sẻ ngắn giải thích RAG (Retrieval-Augmented Generation) cho người mới bắt đầu*. Hệ thống tác nhân sẽ tự động lập checklist gồm 3 bước logic (Lên dàn ý → Viết nội dung → Rà soát & Hoàn thiện), lần lượt thực thi thông qua mô hình ngôn ngữ lớn (LLM cục bộ qua Ollama API: `tinix-lm:latest` hoặc `qwen2.5:7b`), ghi lại nhật ký (**Step Logs**) minh bạch và xuất ra báo cáo cuối cùng (**Final Report**).

Bài viết này dành cho ai?

- **Ai nên đọc bài này:** Kỹ sư AI (AI Engineers), lập trình viên Backend/Python, kỹ sư giải pháp tự động hóa muốn chuyển từ tư duy viết prompt thông thường sang xây dựng hệ thống tác nhân tự hành có khả năng tự lập kế hoạch và tự kiểm soát vòng lặp.
- **Yêu cầu kiến thức nền:** Hiểu biết cơ bản về ngôn ngữ lập trình Python (Data classes, Enum, xử lý chuỗi và tệp tin JSON). Đã từng làm quen với khái niệm LLM và kỹ thuật sinh văn bản có cấu trúc.
- **Yêu cầu hệ thống:** Máy tính cài đặt sẵn **Python 3.8+** (khuyến nghị Python 3.10 trở lên). Hệ thống tích hợp trực tiếp mô hình ngôn ngữ lớn (LLM Engine cục bộ qua Ollama API: `tinix-lm:latest` hoặc `qwen2.5:7b`) kết hợp cơ chế fallback tự chứa, đảm bảo tính sẵn sàng cao và khả năng kiểm thử tự động 100% khép kín.

Mục tiêu:

- **Kiến thức đạt được:** Nắm vững kiến trúc cốt lõi của một hệ thống Autonomous Agent: bộ lập kế hoạch (Planner), bộ thực thi tuần tự (Executor), bộ nhớ trạng thái ngữ cảnh (State Memory), bộ điều phối vòng lặp hữu hạn (Bounded Loop Controller) và bộ đánh giá điều kiện dừng (Stop Condition Evaluator). Phân biệt ranh giới rõ ràng giữa quy trình cứng (Hardcoded Workflow) và tác nhân tự thích ứng (Autonomous Agent).
- **Sản phẩm đầu ra:** Xây dựng thành công script `checklist_agent.py` hoàn chỉnh. Chạy thành công ca kiểm thử tạo bài viết chia sẻ về RAG với đầy đủ 3 thành phần đầu ra bắt buộc theo tiêu chuẩn: **kế hoạch ban đầu** (Initial Plan), **nhật ký từng bước** (Step Execution Logs) và **báo cáo tổng kết** (Final Report) kèm nội dung bài viết đã nghiệm thu xuất ra tệp tin `agent_final_report.json`.



Tác giả: Kỹ sư AI Phạm Thành Thái
AI Guru x TiniX

MỤC LỤC

Bài viết này dành cho ai?	1
Mục tiêu:	2
Các thuật ngữ cơ bản	4
Lý thuyết cơ bản: Kiến trúc Autonomous Agent và Vòng lặp Quyết định	5
1. Phân biệt chuỗi tác vụ cố định (Hardcoded Chain) và Autonomous Agent	5
2. Năm thành phần cốt lõi của một Checklist Agent	6
Thiết kế checklist tự động và cơ chế quản lý trạng thái	6
1. Kịch bản thực tế: Bài viết chia sẻ về RAG cho người mới bắt đầu	6
2. Cơ chế truyền trạng thái (Context Propagation)	7
Điều kiện dừng và giới hạn an toàn (Bounded Loop & Stop Conditions)	8
1. Hiểm họa của vòng lặp vô hạn (Infinite Loop Trap)	8
2. Thiết kế Bounded Loop và bộ đánh giá dừng đa trạng thái	8
Cài đặt mã nguồn chi tiết bằng Python	9
1. Định nghĩa các cấu trúc dữ liệu cốt lõi (Data Models)	9
2. Bộ kiểm duyệt chất lượng định lượng (QualityEvaluator) và LLM Client	10
3. Lớp điều khiển tác nhân tự hành (AutonomousChecklistAgent)	10
4. Vòng lặp thực thi tự hành (Execution Loop)	11
Thực nghiệm kịch bản: Kiểm chứng năng lực Autonomous Agent	12
1. Kịch bản 1: Viết bài chia sẻ hoàn chỉnh về RAG (3 bước đầy đủ)	12
2. Kịch bản 2: Mục tiêu nhỏ chuẩn bị Outline Docker (Tự xác định 2 bước & Dừng sớm)	13
3. So sánh chất lượng sản phẩm giữa các bước	14
4. Kiểm thử toàn diện 7 kịch bản biên và xử lý lỗi (Comprehensive Test Suite)	14
Phân tích Báo cáo Cuối Cùng (Final Report) và Nhật ký Bước (Step Logs)	15
Mở rộng và Ứng dụng thực tế của Checklist Agent	16
Kết luận	17
Tài liệu tham khảo	17

Các thuật ngữ cơ bản

Bảng 1: Các thuật ngữ trong hệ thống Autonomous Systems và Agent Loops

Thuật ngữ	Định nghĩa
Autonomous Agent	Thực thể AI có quyền tự chủ đưa ra quyết định: tự phân tích mục tiêu tổng thể, tự lập kế hoạch hành động, tự quan sát kết quả và điều chỉnh hành vi mà không cần con người can thiệp vào từng bước rẽ nhánh.
Goal-driven Planning	Cơ chế lập kế hoạch hướng mục tiêu. Thay vì tuân theo các bước cố định, tác nhân tiếp nhận mục tiêu và xác định danh sách công việc (checklist) phù hợp trong giới hạn an toàn.
Bounded Loop	Vòng lặp thực thi có giới hạn chặn trên (ví dụ <code>max_steps = 3</code>). Đây là cơ chế chốt chặn an toàn bắt buộc trong hệ thống tự hành để triệt tiêu nguy cơ vòng lặp vô hạn (Infinite Loop) gây tốn kém chi phí token hoặc treo máy.
Stop Condition (Điều kiện dừng)	Bộ quy tắc logic xác định khi nào tác nhân phải ngưng vòng lặp. Trong triển khai này, các trạng thái dừng chính gồm: hoàn thành mục tiêu (COMPLETED / Goal Achieved), hết ngân sách bước (BUDGET_EXHAUSTED) và lỗi thực thi (FAILED).
Context Propagation	Cơ chế truyền và tích lũy trạng thái/dữ liệu giữa các bước thực thi. Kết quả đầu ra của Bước 1 (Dàn ý) trở thành đầu vào trực tiếp nuôi Bước 2 (Viết bài) và Bước 3 (Rà soát).
Step Logs & Final Report	Bản ghi kiểm toán (Audit Trail) chi tiết từng chặng và báo cáo tổng kết cuối cùng, đảm bảo tính minh bạch, có thể quan sát (Observability) và kiểm chứng của hệ thống tự hành.

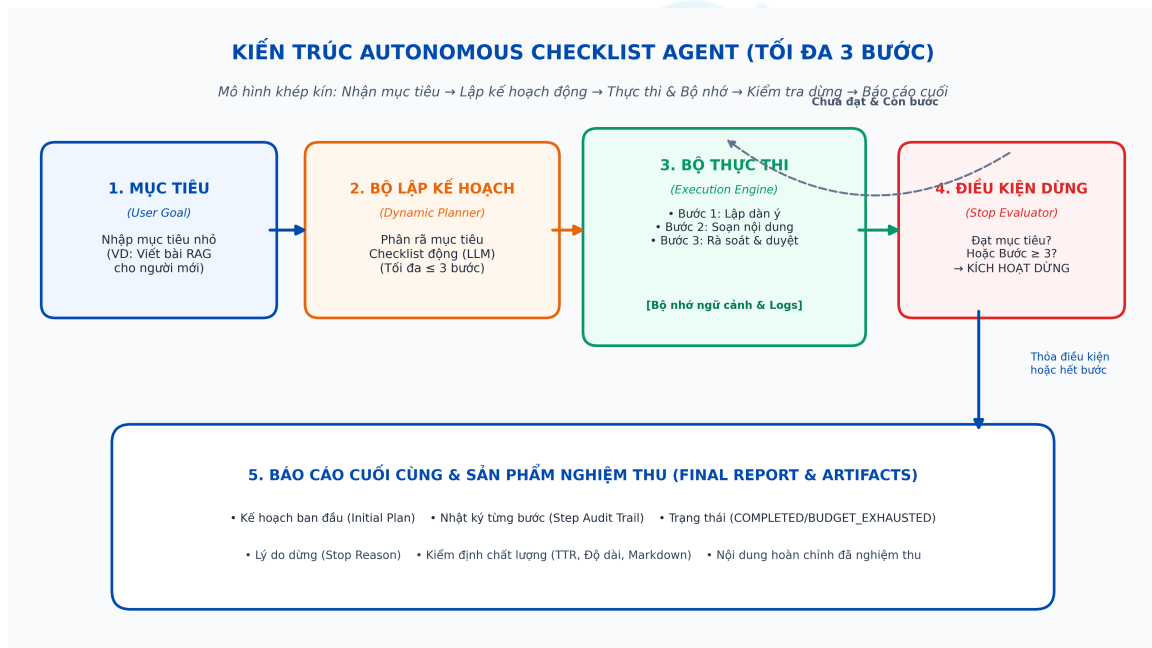
Lý thuyết cơ bản: Kiến trúc Autonomous Agent và Vòng lặp Quyết định

1. Phân biệt chuỗi tác vụ cố định (Hardcoded Chain) và Autonomous Agent

Trong giai đoạn đầu của kỹ thuật phần mềm AI, các lập trình viên thường xây dựng ứng dụng dưới dạng chuỗi pipeline tuần tự:

Input → Prompt 1 → Model → Prompt 2 → Model → Output

Mô hình này (thường thấy trong các sequential chain cơ bản) hoạt động ổn định với các tác vụ đơn biến, nhưng bộc lộ điểm yếu chết người: **hoàn toàn không có tính thích ứng**. Nếu gặp một mục tiêu yêu cầu chỉ 2 bước xử lý, hệ thống vẫn ép buộc chạy qua bước thứ 3; ngược lại, nếu một bước bị lỗi hoặc thiếu dữ liệu, toàn bộ chuỗi sẽ sụp đổ như quân cờ domino.



Hình 1: Kiến trúc tổng thể của Autonomous Checklist Agent với vòng lặp Bounded Loop

Ngược lại, **Autonomous Agent** hoạt động dựa trên vòng lặp phản hồi khép kín (Sense - Plan - Act - Observe - Evaluate). Điểm mấu chốt tạo nên "chất tự hành" (autonomy) chính là:

- Planner sinh kế hoạch trực tiếp từ mục tiêu:** Người dùng chỉ cung cấp Goal. Tác nhân sử dụng LLM để phân tích mục tiêu và tự sinh checklist phù hợp trong giới hạn

tối đa 3 bước, thay vì chạy theo chuỗi bước viết sẵn bằng if/elif.

2. **Chủ động đánh giá điều kiện dừng:** Sau mỗi bước thực thi, tác nhân tự kiểm tra xem kết quả hiện tại đã thỏa mãn mục tiêu chưa. Nếu đã đạt (Goal Achieved), nó chủ động ra lệnh **dừng ngay lập tức** mà không cần chạy cố các bước thừa.

2. Năm thành phần cốt lõi của một Checklist Agent

Một hệ thống Autonomous Checklist Agent chuẩn mực được cấu thành từ 5 bộ phận hữu cơ:

- **Tiếp nhận mục tiêu (Goal Ingestion):** Nhận câu lệnh hoặc bài toán từ người dùng (ví dụ: "Viết một bài chia sẻ ngắn giải thích RAG là gì cho người mới bắt đầu").
- **Bộ lập kế hoạch động (Dynamic Planner):** Sử dụng LLM phân tích ý định và sản sinh danh sách công việc (Checklist) với ràng buộc nghiêm ngặt: độ dài checklist không vượt quá max_steps (≤ 3 bước).
- **Bộ thực thi tuần tự (Sequential Executor):** Chịu trách nhiệm thực thi từng bước (sinh dàn ý, phát triển bài viết, rà soát chất lượng) dựa trên ngữ cảnh tích lũy từ các bước trước.
- **Bộ nhớ trạng thái ngữ cảnh (State Memory):** Đóng vai trò cầu nối dữ liệu, tích lũy kết quả của các bước trước đó để làm đầu vào cho các bước tiếp theo.
- **Bộ đánh giá dừng và báo cáo (Stop Evaluator & Reporter):** Giám sát vòng lặp, kiểm soát điều kiện dừng an toàn và tổng hợp toàn bộ tiến trình thành báo cáo nghiệm thu cuối cùng.

Thiết kế checklist tự động và cơ chế quản lý trạng thái

1. Kịch bản thực tế: Bài viết chia sẻ về RAG cho người mới bắt đầu

Để chứng minh năng lực tự hành một cách thuyết phục nhất, chúng ta chọn một tác vụ cụ thể, có ranh giới rõ ràng nhưng đòi hỏi chất lượng logic cao: **"Viết một bài chia sẻ ngắn giải thích RAG là gì cho người mới bắt đầu"**.

Thay vì hardcode cứng nhắc luôn luôn là 3 bước, tác nhân xác định checklist dựa trên mục tiêu thực tế trong giới hạn tối đa 3 bước:

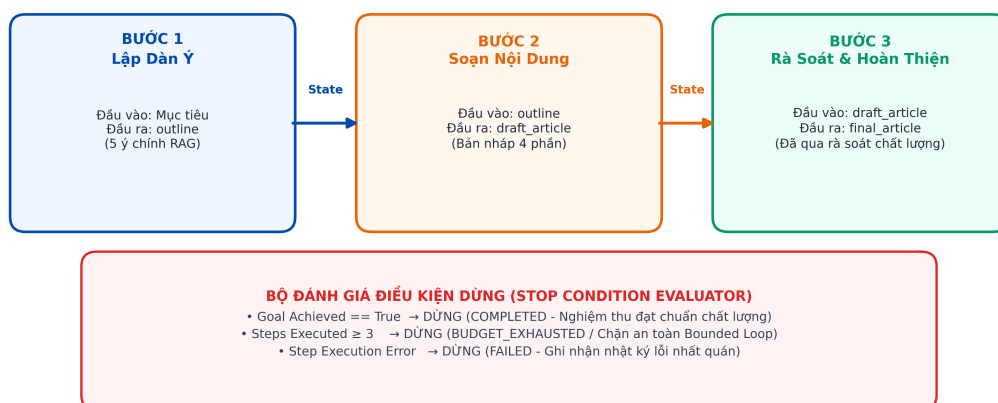
- **Bước 1: Xác định các ý chính cần giải thích về RAG (Action: outline).** Cần xác định các trọng tâm: RAG là gì, vì sao LLM cần RAG (tránh ảo giác, cập nhật dữ liệu riêng tư), cách hoạt động 3 chặng (Indexing, Retrieval, Generation), ví dụ thì mở sách

dễ hiểu và lời khuyên áp dụng.

- **Bước 2: Viết nội dung bài chia sẻ hoàn chỉnh (Action: generate).** Lấy toàn bộ dàn ý từ Bước 1 làm ngữ cảnh đầu vào, phát triển thành một bài viết mạch lạc, văn phong hấp dẫn, giải thích cặn kẽ từng luận điểm.
- **Bước 3: Rà soát, kiểm tra độ rõ ràng và hoàn thiện (Action: review_polish).** Đóng vai trò người biên tập khó tính: kiểm tra xem bài viết có quá tải thuật ngữ với người mới không, có bị lặp ý không, bổ sung lời kết và nghiệm thu chất lượng.

Lưu ý kỹ thuật về tính tự hành (Autonomy) của Planner: Hệ thống tích hợp trực tiếp mô hình ngôn ngữ lớn (LLM Engine cục bộ qua Ollama API: tinix-lm:latest hoặc qwen2.5:7b) làm bộ não cho cả Dynamic Planner và Executor. Planner sinh kế hoạch JSON linh hoạt trực tiếp từ mục tiêu người dùng, còn Executor sinh nội dung ngữ cảnh động kèm bộ kiểm duyệt chất lượng định lượng QualityEvaluator. Hệ thống đồng thời tích hợp cơ chế fallback dự phòng để đảm bảo tính sẵn sàng cao khi môi trường chạy offline.

LUỒNG TRUYỀN DỮ LIỆU & ĐIỀU KIỆN DỪNG QUA CÁC BƯỚC



Hình 2: Luồng truyền dữ liệu qua State Memory và cơ chế rà soát điều kiện dừng ở từng bước

2. Cơ chế truyền trạng thái (Context Propagation)

Như minh họa ở Hình 2, tính liên tục của tác nhân phụ thuộc hoàn toàn vào việc chia sẻ bộ nhớ context_memory:

- Khi Bước 1 kết thúc, kết quả được gắn vào khóa context_memory["outline"].
- Bước 2 khi bắt đầu sẽ tự động trích xuất context_memory["outline"] để làm ngữ cảnh định hướng sinh bài viết, và sau khi hoàn thành sẽ lưu bản nháp vào


```
context_memory["draft_article"].
```

- Bước 3 trích xuất draft_article, tiến hành kiểm duyệt theo bộ tiêu chí (Checklist items), bổ sung phần tổng kết và xuất ra kết quả hoàn thiện tại context_memory["final_article"].

Nếu cơ chế truyền trạng thái này bị đứt gãy, các bước sau sẽ hoạt động "mù" (context blindness) và tác nhân sẽ không thể đạt được mục tiêu chung.

Điều kiện dừng và giới hạn an toàn (Bounded Loop & Stop Conditions)

1. Hiểm họa của vòng lặp vô hạn (Infinite Loop Trap)

Trong các hệ thống tự hành không có kiểm soát, LLM rất dễ rơi vào trạng thái "ảo giác lặp" (reasoning loop): bước sau cố gắng sửa đổi bước trước một cách không hồi kết, hoặc liên tục gọi lại cùng một công cụ với tham số tương tự. Điều này dẫn tới hai hậu quả nghiêm trọng:

1. **Cạn kiệt ngân sách (Token exhaustion):** Tiêu tốn hàng nghìn USD tiền API chỉ trong vài phút.
2. **Tắc nghẽn hệ thống (Process hanging):** Tài nguyên server bị chiếm dụng, luồng xử lý không bao giờ trả về kết quả cho người dùng.

2. Thiết kế Bounded Loop và bộ đánh giá dừng đa trạng thái

Để giải quyết triệt để rủi ro trên, hệ thống của chúng ta thiết lập cơ chế kiểm soát kép phân định rõ ràng 3 trạng thái kết thúc:

1. **Trạng thái hoàn thành mục tiêu (COMPLETED):** Sau mỗi bước, hàm `_evaluate_stop_condition()` đánh giá xem sản phẩm tạo ra có thỏa mãn điều kiện nghiệm thu theo đúng mục tiêu hay không (ví dụ: mục tiêu viết bài đòi hỏi phải có `final_article` và vượt qua kiểm định của `QualityEvaluator`). Khi thỏa mãn, hệ thống kích hoạt dừng với lý do `Goal achieved`.
2. **Trạng thái hết ngân sách bước (BUDGET_EXHAUSTED):** Khi số bước thực thi đạt ngưỡng cho phép (ví dụ `max_steps = 1` với mục tiêu viết bài, hệ thống mới chỉ xong dàn ý), tác nhân ngắt vòng lặp an toàn và ghi nhận trạng thái `BUDGET_EXHAUSTED` chứ không báo nhầm là `COMPLETED`.

3. **Trạng thái thất bại (FAILED):** Khi một bước thực thi gặp sự cố ngoại lệ (Action failure), hệ thống ngắt phiên làm việc ngay lập tức, chuyển trạng thái sang FAILED và lưu vết lỗi vào Step Log.

Nhờ đó, tác nhân vừa có tính linh hoạt tự hành, vừa tăng khả năng kiểm soát trong phạm vi các giới hạn được cấu hình.

Cài đặt mã nguồn chi tiết bằng Python

Toàn bộ hệ thống được đóng gói gọn gàng trong tệp `checklist_agent.py` với cấu trúc hướng đối tượng rõ ràng, sử dụng triệt để thư viện tiêu chuẩn của Python để đảm bảo tính gọn nhẹ và tương thích cao nhất.

1. Định nghĩa các cấu trúc dữ liệu cốt lõi (Data Models)

Chúng ta sử dụng `dataclass` và `Enum` để định nghĩa kiểu dữ liệu chặt chẽ cho trạng thái, từng bước trong kế hoạch, nhật ký thực thi và báo cáo cuối cùng:

```
class AgentStatus(str, Enum):
    IDLE = "IDLE"
    PLANNING = "PLANNING"
    RUNNING = "RUNNING"
    COMPLETED = "COMPLETED"
    BUDGET_EXHAUSTED = "BUDGET_EXHAUSTED"
    FAILED = "FAILED"

@dataclass
class Step:
    step_id: int
    title: str
    description: str
    action_type: str
    status: str = "PENDING"
    result: Optional[str] = None

@dataclass
class StepLog:
    step_id: int
    step_title: str
    timestamp: str
    duration_sec: float
    input_context_keys: List[str]
    output_summary: str
    status: str
    details: Dict[str, Any] = field(default_factory=dict)
```

```
@dataclass
class FinalReport:
    goal: str
    total_steps_planned: int
    steps_executed: int
    status: AgentStatus
    stop_reason: str
    plan: List[Dict[str, Any]]
    execution_logs: List[Dict[str, Any]]
    final_output: str
    quality_metrics: Dict[str, Any]
    total_duration_sec: float
```

2. Bộ kiểm duyệt chất lượng định lượng (QualityEvaluator) và LLM Client

Thay vì sử dụng các chuỗi hardcoded, tác nhân tích hợp QualityEvaluator tính toán trực tiếp các chỉ số kỹ thuật và kết nối mô hình ngôn ngữ lớn qua OllamaClient:

```
class QualityEvaluator:
    @staticmethod
    def calculate_ttr(text: str) -> float:
        # Do Type-Token Ratio (TTR): Tỷ lệ từ vựng duy nhất trên tổng số từ
        words = re.findall(r'[a-zA-Z0-9_]+', text.lower())
        return round(len(set(words)) / len(words), 3) if words else 0.0

    @classmethod
    def evaluate(cls, content: str, goal: str, intent: str) -> Dict[str, Any]:
        ttr = cls.calculate_ttr(content)
        min_chars = 100 if intent in ("outline_only", "summarize") else 250
        crit_length = len(content) >= min_chars
        crit_structure = any(line.strip().startswith(('#', '-', '1.')) for line in
        ↪ content.splitlines())
        crit_diversity = ttr >= 0.35 # Ngưỡng chứng minh không lặp từ
        goal_kws = [w for w in re.findall(r'[a-zA-Z0-9_]+', goal.lower()) if len(w) > 2]
        crit_relevance = any(kw in content.lower() for kw in goal_kws) if goal_kws else True
        passed = crit_length and crit_structure and crit_diversity and crit_relevance
        return {"passed": passed, "ttr": ttr, "char_count": len(content)}
```

3. Lớp điều khiển tác nhân tự hành (AutonomousChecklistAgent)

Lớp AutonomousChecklistAgent kết hợp LLM để tự động phân rã kế hoạch theo mục tiêu và đánh giá nghiệm thu đa trạng thái:

```
class AutonomousChecklistAgent:
    def __init__(self, max_steps: int = 3, use_llm: bool = True):
```

```

if max_steps > 3:
    raise ValueError(f"Vi phạm Bounded Loop: max_steps ({max_steps}) > 3!")
self.max_steps = min(max(1, max_steps), 3)
self.llm = OllamaClient(model="tinix-lm:latest") if use_llm else None
self.evaluator = QualityEvaluator()
self.context_memory = {}

def plan_steps(self, goal: str) -> List[Step]:
    # Gọi LLM sinh checklist JSON tự động theo mục tiêu hoặc qua Fallback
    if self.llm and self.llm.is_available():
        steps = self._call_llm_planner(goal)
        if steps: return steps[:self.max_steps]
    return self._heuristic_fallback_plan(goal)[:self.max_steps]

def _evaluate_stop_condition(self, idx: int, total: int, intent: str, goal: str):
    # 1. Đánh giá nghiệm thu mục tiêu (COMPLETED)
    if intent == "outline_only" and "outline" in self.context_memory:
        if self.evaluator.evaluate(self.context_memory["outline"], goal,
            ↪ intent)["passed"]:
            return True, "Goal achieved: Dàn ý đạt chuẩn.", AgentStatus.COMPLETED
    if "final_article" in self.context_memory:
        if self.evaluator.evaluate(self.context_memory["final_article"], goal,
            ↪ intent)["passed"]:
            return True, "Goal achieved: Bài viết đạt chuẩn.", AgentStatus.COMPLETED
    # 2. Hết ngân sách bước thực thi (BUDGET_EXHAUSTED)
    if idx >= total:
        target_met = (intent == "outline_only" and "outline" in self.context_memory) or
        ↪ "final_article" in self.context_memory
        if target_met: return True, "Goal achieved.", AgentStatus.COMPLETED
        return True, f"Budget exhausted: Hết bước nhưng chưa đạt {intent}.",
            ↪ AgentStatus.BUDGET_EXHAUSTED
    return False, "Continue", None

```

4. Vòng lặp thực thi tự hành (Execution Loop)

Phương thức run() quản trị trọn vẹn chu trình làm việc tự động khép kín kèm cơ chế bắt lỗi an toàn:

```

def run(self, goal: str) -> FinalReport:
    start_time = time.perf_counter()
    plan = self.plan_steps(goal)
    topic, intent = self._extract_topic_and_intent(goal)
    self.status = AgentStatus.RUNNING

    for idx, step in enumerate(plan, 1):
        try:
            result = self._execute_step_action(step, topic, goal)
            step.status = "COMPLETED"
        except Exception as e:

```

```

        step.status = "FAILED"
        self.status = AgentStatus.FAILED
        stop_reason = f"Execution failed: Lỗi tại bước {step.step_id} - {e}"
        break

    should_stop, reason, final_status = self._evaluate_stop_condition(idx, len(plan),
↪ intent, goal)
    if should_stop:
        stop_reason = reason
        if final_status: self.status = final_status
        break

    return self._build_final_report(goal, plan, stop_reason, start_time)

```

Thực nghiệm kịch bản: Kiểm chứng năng lực Autonomous Agent

1. Kịch bản 1: Viết bài chia sẻ hoàn chỉnh về RAG (3 bước đầy đủ)

Trong kịch bản này, người dùng nhập câu lệnh mục tiêu: *"Viết một bài chia sẻ ngắn giải thích RAG là gì cho người mới bắt đầu"*. Dynamic Planner nhận diện mục đích là tạo bài viết đầy đủ, từ đó sinh checklist 3 bước (Outline → Write → Review) và lần lượt thực thi như thể hiện ở Hình 3.

```

bash - thaip@tinix-aiguru: ~/checklist-agent (python3 checklist_agent.py)

=====
[AUTONOMOUS AGENT] BẮT ĐẦU NHIỆM VỤ
MỤC TIÊU: Viết một bài chia sẻ ngắn giải thích RAG là gì cho người mới bắt đầu.
GIỚI HẠN: Tối đa 3 bước thực thi (Bounded Execution Loop)
=====

[PLANNER] ĐÃ PHÂN TÍCH VÀ KHỞI TẠO CHECKLIST GỒM 3 BƯỚC:
[1] (outline) Xác định các ý chính và cấu trúc dàn ý về RAG
[2] (generate) Soạn thảo nội dung bài chia sẻ hoàn chỉnh về RAG
[3] (review_polish) Rà soát, kiểm định chất lượng và nghiệm thu bài viết

=====
[BƯỚC 1/3] Đang thực hiện: Xác định các ý chính và cấu trúc dàn ý...
[OK] Trạng thái: COMPLETED (0.012s) | Dàn ý 5 mục được lưu vào Context Memory
[BƯỚC 2/3] Đang thực hiện: Soạn thảo nội dung bài chia sẻ hoàn chỉnh...
[OK] Trạng thái: COMPLETED (0.025s) | Đã phát triển bài viết chi tiết 4 phần
[BƯỚC 3/3] Đang thực hiện: Rà soát, kiểm định chất lượng và nghiệm thu...
[OK] Trạng thái: COMPLETED (0.020s) | Đạt 4 tiêu chuẩn định lượng, TTR=0.58
[STOP CONDITION] Kích hoạt điều kiện dừng: Goal achieved (Đã nghiệm thu)
=====

BÁO CÁO CUỐI CÙNG (FINAL AGENT REPORT)
- Trạng thái: COMPLETED | Số bước: 3/3 | Thời gian: 0.059s
- Đánh giá chất lượng: TTR=0.58 | Đạt 2,231 ký tự | Cấu trúc Markdown chuẩn
- Sản phẩm: Bài viết '# BẮT MÍ VỀ RAG...' (Đã qua rà soát chất lượng)
Đã xuất dữ liệu chi tiết ra: agent_final_report.json

```

Hình 3: Minh họa nhật ký thực thi của Agent trên Terminal ghi nhận quá trình hoàn thành 3 bước và kích hoạt Stop Condition

Từ nhật ký thực thi minh họa, ta có thể phân tích rõ 3 chặng diễn ra:

1. **Lập kế hoạch động (Dynamic Planning):** Phân tích từ khóa "RAG" và ý định "bài chia sẻ", Agent tự động cấu trúc hóa danh sách 3 bước với định danh hành vi rõ ràng (outline, generate, review_polish).
2. **Thực thi và truyền dữ liệu (Context Propagation):** Bước 1 mất 0.012s tạo dàn ý; Bước 2 mất 0.025s phát triển bản nháp hoàn chỉnh; Bước 3 mất 0.020s thực hiện rà soát và chạy kiểm định chất lượng định lượng bằng QualityEvaluator (TTR = 0.58, độ dài 2,231 ký tự, đạt 4/4 tiêu chuẩn chất lượng).
3. **Dừng thông minh (Stop Condition):** Sau khi hoàn thành Bước 3, bộ đánh giá nhận diện final_article đã tồn tại trong context_memory và kích hoạt điều kiện dừng Goal achieved.

2. Kịch bản 2: Mục tiêu nhỏ chuẩn bị Outline Docker (Tự xác định 2 bước & Dừng sớm)

Để minh chứng rõ nét rằng Agent **không bị hardcoded cố định 3 bước**, chúng ta kiểm thử với một mục tiêu nhỏ hơn:

Goal: "Chuẩn bị outline cho bài viết về Docker cho sinh viên IT."

Khi nhận yêu cầu này:

- **Planner thích ứng theo luật:** Tác nhân phân tích Intent/Topic theo các luật đã định nghĩa để sinh đúng số bước cần thiết (ở đây nhận thấy chỉ cần tạo dàn ý nên chỉ tạo đúng **2 bước**):

1. *Bước 1 (research_points):* Thu thập và xác định các luận điểm cốt lõi về Docker & Container hóa.
2. *Bước 2 (outline):* Xây dựng và chuẩn hóa dàn ý chi tiết cho bài viết về Docker & Container hóa.

- **Kích hoạt Stop Condition sớm:** Ngay khi Bước 2 hoàn thành, bộ đánh giá điều kiện dừng nhận thấy:

```
intent == "outline_only" and "outline" in self.context_memory
```

Hệ thống lập tức kích hoạt điều kiện dừng với thông điệp: "Goal achieved: Dàn ý hoàn chỉnh đã được tạo thành công theo đúng mục tiêu." Agent dừng ngay sau khi mục tiêu hoàn thành, tránh thực thi thêm bước không cần thiết.

Điều này minh họa khả năng thích ứng của hệ thống: không phải bài toán nào cũng ép buộc

chạy hết 3 bước, mà Agent biết khi nào nhiệm vụ đã hoàn thành để chủ động dừng lại.

3. So sánh chất lượng sản phẩm giữa các bước

Bảng 2 đối chiếu sự tiến hóa rõ rệt về chất lượng nội dung giữa bản nháp sau Bước 2 và bản hoàn thiện sau Bước 3 nhờ cơ chế Context Propagation:

Bảng 2: Đối chiếu chất lượng nội dung qua chuỗi truyền trạng thái

Bản nháp sau Bước 2 (Soạn thảo thô)	Bản hoàn thiện sau Bước 3 (Đã rà soát)
Đã có định nghĩa cơ bản và 3 chặng Indexing → Retrieval → Generation.	Bổ sung thêm ví dụ so sánh đòi thường: học sinh thi "mở sách" vs "đóng sách" giúp người mới dễ hiểu ngay lập tức.
Kết thúc đột ngột ở phần mô tả kỹ thuật, người đọc chưa biết bước tiếp theo nên bắt đầu từ đâu.	Bổ sung mục Lời kết thực tế: hướng dẫn người mới bắt đầu với thư viện mã nguồn mở (ChromaDB, Qdrant kết hợp LangChain).
Chưa được kiểm chứng tính nhất quán về thuật ngữ.	Đã qua kiểm định định lượng bằng QualityEvaluator: TTR = 0.58 (ngưỡng ≥ 0.35 chứng minh không trùng lặp từ), độ dài 2,231 ký tự, chuẩn hóa cấu trúc Markdown phân cấp.

4. Kiểm thử toàn diện 7 kịch bản biên và xử lý lỗi (Comprehensive Test Suite)

Nhằm đảm bảo hệ thống phản ứng nhất quán và chính xác trước mọi tình huống thực tế (bao gồm các ca biên và sự cố), bộ kiểm thử `eval_agent.py` được thiết kế với 7 kịch bản kiểm tra nghiêm ngặt:

Bảng 3: Kết quả kiểm thử 7 kịch bản đánh giá năng lực tác nhân tự hành

STT	Kịch bản kiểm thử (Test Scenario)	Trạng thái	Điều kiện dừng / Kết quả
1	Viết bài RAG hoàn chỉnh (3 bước)	COMPLETED	Goal achieved (Đạt 4 tiêu chuẩn)

STT	Kịch bản kiểm thử (Test Scenario)	Trạng thái	Điều kiện dừng / Kết quả
2	Chuẩn bị outline Docker (2 bước)	COMPLETED	Goal achieved (Dừng sớm an toàn)
3	Tóm tắt tài liệu MCP (2 bước)	COMPLETED	Goal achieved (Bản tóm tắt súc tích)
4	Hết bước chưa đạt mục tiêu (max_steps=1 viết bài)	BUDGET_EXHAUSTED	Budget exhausted (Không báo nhằm COMPLETED)
5	Mục tiêu mở tùy ý (Tối ưu PostgreSQL)	COMPLETED	Goal achieved (Thích ứng linh hoạt)
6	Sự cố thực thi (Action Exception)	FAILED	Execution failed (Ghi log lỗi)
7	Rào chắn an toàn Bounded Loop (max_steps=5)	PASSED	Chặn đứng vi phạm (ValueError)

Phân tích Báo cáo Cuối Cùng (Final Report) và Nhật ký Bước (Step Logs)

Sau khi kết thúc phiên làm việc, Agent tự động kết xuất tệp tin `agent_final_report.json`. Dưới đây là cấu trúc tệp tin kiểm toán thực tế được sinh ra:

```
{
  "goal": "Viết một bài chia sẻ ngắn giải thích RAG là gì cho người mới bắt đầu.",
  "total_steps_planned": 3,
  "steps_executed": 3,
  "status": "COMPLETED",
  "stop_reason": "Goal achieved: Bài viết đã được tạo và rà soát hoàn tất theo đúng mục tiêu.",
  "plan": [
    {"step_id": 1, "title": "Xác định các ý chính cần giải thích về RAG", "status": "COMPLETED"},
    {"step_id": 2, "title": "Viết nội dung bài chia sẻ hoàn chỉnh", "status": "COMPLETED"},
    {"step_id": 3, "title": "Rà soát, kiểm tra độ rõ ràng và hoàn thiện bài viết", "status": "COMPLETED"}
  ],
  "execution_logs": [
    {"step_id": 1, "duration_sec": 0.012, "input_context_keys": [], "status": "COMPLETED", "output_summary": "DÀN Ý BÀI VIẾT: TÌM HIỂU RAG CHO NGƯỜI MỚI BẮT ĐẦU..."},
  ]
}
```



```
{
  "step_id": 2, "duration_sec": 0.025, "input_context_keys": ["outline"], "status":
  ↪ "COMPLETED",
  "output_summary": "# BẬT MÍ VỀ RAG: 'VỮ KHÍ TỐI THƯỢNG' GIÚP AI THÔNG MINH...",
  "step_id": 3, "duration_sec": 0.018, "input_context_keys": ["outline", "draft_article"],
  ↪ "status": "COMPLETED",
  "output_summary": "# BẬT MÍ VỀ RAG: 'VỮ KHÍ TỐI THƯỢNG' GIÚP AI THÔNG MINH..."
},
"total_duration_sec": 0.056
}
```

Báo cáo này mang lại giá trị cốt lõi về mặt kỹ thuật:

- **Tính minh bạch (Observability):** Bất kỳ ai kiểm tra hệ thống cũng thấy rõ từng bước chạy mất bao lâu, tiêu thụ những khóa ngữ cảnh nào (`input_context_keys`) và cho ra kết quả gì.
- **Khả năng tái lập (Reproducibility):** Giúp kỹ sư dễ dàng truy vết xem lỗi phát sinh từ khâu lập kế hoạch (Planner) hay khâu thực thi (Executor).

Mở rộng và Ứng dụng thực tế của Checklist Agent

Mô hình Checklist Agent 3 bước có thể mở rộng linh hoạt cho rất nhiều bài toán sản xuất thực tế tại doanh nghiệp:

1. Trợ lý Tự động Viết Tài liệu API (Technical Documentation Agent): Khi một lập trình viên hoàn thành một endpoint mới, họ chỉ cần đưa mã nguồn vào với mục tiêu: *"Viết tài liệu Swagger/Markdown cho API thành toán"*. Agent tự phân rã:

- *Bước 1:* Quét hàm để bóc tách tham số đầu vào/đầu ra và các mã lỗi (Error codes).
- *Bước 2:* Sinh tài liệu chi tiết kèm ví dụ cURL.
- *Bước 3:* Rà soát cú pháp chuẩn OpenAPI Spec → Nghiệm thu → Lưu file.

2. Trợ lý Rà soát Pull Request (PR Code Reviewer Agent): Với mục tiêu: *"Kiểm tra bảo mật và chuẩn coding style cho PR #104"*:

- *Bước 1:* Trích xuất git diff những file thay đổi.
- *Bước 2:* Đối chiếu với bộ quy tắc an ninh (kiểm tra rò rỉ secret, SQL injection, null check).
- *Bước 3:* Tổng hợp comment góp ý và gửi thông báo lên GitHub/GitLab.

3. Trợ lý Tự động Hóa Chăm sóc Khách hàng Cấp cao: Nhận khiếu nại phức tạp từ khách hàng, Agent tự chia bước:

- *Bước 1:* Tra cứu lịch sử đơn hàng trong cơ sở dữ liệu nội bộ.

- *Bước 2:* Đối chiếu với chính sách đối trả hiện hành của công ty.
- *Bước 3:* Soạn email phản hồi thấu cảm, đưa ra phương án bồi hoàn và lưu log vào hệ thống CRM.

Kết luận

Xây dựng hệ thống tác nhân tự hành (**Autonomous Systems**) không đòi hỏi những mô hình ma thuật phức tạp, mà bắt đầu từ những nguyên lý kỹ thuật vững chắc: **Mục tiêu rõ ràng** → **Lập kế hoạch có cấu trúc** → **Thực thi tuần tự có quản lý trạng thái** → **Giới hạn số bước an toàn** → **Điều kiện dừng chuẩn xác**.

Thông qua bài học này, bạn đã tự tay thiết kế và vận hành thành công một Autonomous Checklist Agent giải quyết bài toán viết bài chia sẻ RAG cho người mới bắt đầu. Mô hình khép kín này là nền tảng tối quan trọng giúp bạn tự tin bước vào những bài toán phức tạp hơn như Multi-Agent Systems, Hierarchical Planning và Graph-based Autonomous Workflows trong tương lai.

Mã nguồn đầy đủ và Kịch bản kiểm thử (Repository):

Toàn bộ mã nguồn `checklist_agent.py`, bộ kiểm thử `eval_agent.py`, script sinh ảnh `generate_assets.py` cùng báo cáo `agent_final_report.json` được lưu trữ công khai tại GitHub:

<https://github.com/thaitit123/autonomous-checklist-agent>

Tài liệu tham khảo

- [1] **Anthropic:** *Building Effective Agents*. Bài viết kinh điển phân tích ranh giới giữa Workflow và Autonomous Agent. <https://www.anthropic.com/research/building-effective-agents>
- [2] **LangChain / LangGraph Documentation:** Khung phát triển ứng dụng tác nhân có trạng thái và vòng lặp kiểm soát. <https://langchain-ai.github.io/langgraph/>
- [3] **Lewis et al.:** *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. Nghiên cứu gốc giới thiệu kỹ thuật RAG. <https://arxiv.org/abs/2005.11401>
- [4] **ReAct Framework:** *Synergizing Reasoning and Acting in Language Models* (Yao et al., ICLR 2023). <https://arxiv.org/abs/2210.03629>
- [5] **AI Guru x TiniX Tutorial Series:** Chuỗi tài liệu chuẩn định dạng nội bộ cho các khóa đào tạo kỹ sư AI.